

SIMATS ENGINEERING
DEPARTMENT OF INFORMATION AND TECHNOLOGY
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

COURSE CODE :	CSA1202	COURSE TITLE :	COMPUTER ARCHITECTURE
CO ASSESSED :	CO2	ASSESSMENT:	ASSESSMENT TOOL 1-ANALYTICAL PROBLEM SOLVING
WEIGHTAGE:	45%	TOTAL MARKS:	25
C02 STATEMENT:	APPLY INTEGER AND FLOATING POINT ARITHMETIC		
STUDENT NAME :	PRATHIPA S	REG NO:	192521419
DEPARTMENT :	B.TECH IT	DATE:	25/07/26

Code of Conduct:

S.Prathipa (192521419)

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary Action.

Signature of the student: *S.prathipa*

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps.

Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and $+5$. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

MARKS ALLOCATION

Criteria	Understanding of problem context (20%)	Applications of concepts (20%)	Problem solving approach (20%)	Accuracy of solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----------	---------------------	----------------------	----------------------	---------------------

Understand ing of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor error	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem- Solving Approach	20	Logical, well- structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well- organized, and properly explained solution	Understandab le with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

SIMATS ENGINEERING
CO1 - ASSESSMENT TOOL 1

Name	Register number	Course code	Course
S.Prathipa	192521419	CSA1202	Computer Architecture

Question 1

A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final binary results.

Step 1: Binary Representation

Representation of +45

Decimal = 45

Binary (8-bit):

45 = 00101101

Since it is positive, this is its 2's complement represent

Representation of -23

First convert +23 into binary.

23 = 00010111

Find 2's complement:

1's complement = **11101000**

11101000

+00000001

= **11101001**

Therefore,

-23 = 11101001

Step 2: Addition (+45 + (-23))

Binary values

00101101 (+45)

11101001 (-23)

00010110

Ignore the carry beyond 8 bits.

Binary result:

00010110

Decimal value = **22**

+45 + (-23) = +22

Overflow Detection

Overflow occurs only when:

- Two positive numbers produce a negative result.
- Two negative numbers produce a positive result.

+45 is positive.

-23 is negative.

Since the operands have opposite signs, **overflow cannot occur**.

Overflow = No

Step 3: Subtraction (+45 - (-23))

Subtracting a negative number is equivalent to addition.

$$+45 - (-23)$$

$$= +45 + 23$$

23 in binary:

00010111

Now perform addition.

00101101

00010111

01000100

Binary result: 01000100

Decimal = **68**

$$+45 - (-23) = 68$$

How Processor Detects Overflow

The processor compares:

- Carry into the MSB
- Carry out of the MSB

If both carries are different, **Overflow = 1**

If both carries are same, **Overflow = 0**

Carry into MSB = Carry out of MSB

Overflow Flag = 0

Final Results

Operation	Result (Binary)	Result (Decimal)	Overflow
+45 + (-23)	00010110	22	no
+45 - (-23)	01000100	68	no

Question 2

A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

Generate and Propagate Signals

For each bit,

Generate Signal (G_i)

$$G_i = A_i \times B_i$$

A carry is generated whenever both input bits are 1.

Propagate Signal (P_i)

$$P_i = A_i \oplus B_i$$

If P_i is 1, any incoming carry is propagated to the next stage.

Example

Consider

$$A = 1011$$

$$B = 0110$$

$$C_{in} = 0$$

Step 1: Generate Signals

Bt	A	B	G=AB
----	---	---	------

0	1	0	0
---	---	---	---

1	1	1	1
---	---	---	---

2	0	1	0
---	---	---	---

3	1	0	0
---	---	---	---

Step 2: Propagate Signals

Bt	A	B	P=AB
----	---	---	------

0	1	0	1
---	---	---	---

1	1	1	0
---	---	---	---

2	0	1	1
---	---	---	---

3	1	0	1
---	---	---	---

Carry Computation

The CLA computes carries using the following equations:

$$C1 = G0 + P0Cin$$

$$C2 = G1 + P1G0 + P1P0Cin$$

$$C3 = G2 + P2G1 + P2P1G0 + P2P1P0Cin$$

$$C4 = G3 + P3G2 + P3P2G1 + P3P2P1G0 + P3P2P1P0Cin$$

Since all carry values are calculated simultaneously, no waiting occurs.

Sum Calculation

The sum bits are calculated using

$$S_i = P_i \oplus C_i$$

Each sum bit is produced immediately after carry computation.

Time Delay Comparison

For a 4-bit Ripple Carry Adder,

$$\text{Total Delay} = \text{Delay of 4 Full Adders} = 4 \times T$$

where T is the delay of one Full Adder.

For a 4-bit CLA,

Carry signals are generated simultaneously.

$$\text{Total Delay} \approx T + \text{Logic Delay}$$

CLA has much lower delay than Ripple Carry Addition.

Question 3

In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and $+5$. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Given Data

- Multiplicand (M) = -6
- Multiplier (Q) = $+5$
- Number of bits = 4

Binary Representation

$$+5 = 0101$$

$$+6 = 0110$$

2's complement of 0110:

$$1's \text{ complement} = 1001$$

$$\text{Add } 1 = 1010$$

Therefore,

$$-6 = 1010$$

Initial Registers:

- Accumulator (A) = 0000

- Multiplier (Q) = 0101
- Booth Bit (Q-1) = 0
- Count = 4

Booth's Algorithm Rules

$Q_0 Q_{-1}$	Operation
00	No operation
01	$A = A + M$
10	$A = A - M$
11	No operation

Step-by-Step Execution

Iteration 1

$$Q_0 Q_{-1} = 10$$

Operation:

$$A = A - M$$

After subtraction:

$$A = 0110$$

Arithmetic Right Shift:

$$A = 0011$$

$$Q = 0010$$

$$Q_{-1} = 1$$

Iteration 2

$$Q_0Q_{-1} = 01$$

Operation:

$$A = A + M$$

Result:

$$A = 1101$$

Arithmetic Right Shift:

$$A = 1110$$

$$Q = 1001$$

$$Q_{-1} = 0$$

Iteration 3

$$Q_0Q_{-1} = 10$$

Operation:

$$A = A - M$$

Result:

$$A = 0100$$

Arithmetic Right Shift:

$$A = 0010$$

$$Q = 0100$$

$$Q_{-1} = 1$$

Iteration 4

$$Q_0Q_{-1} = \mathbf{01}$$

Operation:

$$A = A + M$$

Result:

$$A = \mathbf{1100}$$

Arithmetic Right Shift:

$$A = \mathbf{1110}$$

$$Q = \mathbf{0010}$$

$$Q_{-1} = \mathbf{0}$$

Final Result

Final Product (AQ):

$$11100010_2$$

Decimal Equivalent:

$$-30$$

Hence,

$$-6 \times +5 = -30$$

Question 4

A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Given Data

Dividend = **13**

Divisor = **3**

Binary Representation:

$13 = 1101_2$

$3 = 0011_2$

Expected Result:

$13 \div 3 = 4$

Quotient = **0100_2**

Remainder = **0001_2**

Step 1

- Shift the remainder and quotient registers to the left.
- Subtract the divisor from the remainder.
- If the result is positive, place **1** in the quotient.

- If the result is negative, restore the previous remainder and place **0** in the quotient.

Step 2

- Repeat the same process for all four bits of the dividend.

Final Output

Quotient = **0100**

Remainder = **0001**

Therefore,

$13 \div 3 = 4$, Remainder = 1

Question 5

A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Given Number

Decimal Number = -13.25

Step 1: Convert Decimal to Binary

Integer Part:

$$13_{10} = 1101_2$$

Fractional Part:

$$0.25_{10} = 0.01_2$$

Therefore,

$$13.25 = 1101.01_2$$

Normalized Form:

$$1.10101 \times 2^3$$

IEEE 754 Single Precision (32-bit)

Single precision uses **32 bits** divided into:

- **1 bit** – Sign
- **8 bits** – Exponent
- **23 bits** – Mantissa (Fraction)

Sign Bit

Since the number is negative,

Sign = 1

Exponent

Actual exponent = **3**

Bias = **127**

Stored exponent = $127 + 3 = 130$

130 in binary = **10000010**

Mantissa

Normalized binary: **1.10101**

The leading **1** is not stored.

Final Single Precision Representation

Sign	Exponent	Mantissa
------	----------	----------

1	10000010	10101000000000000000 0000
---	----------	------------------------------